

Sesión 11

Diseñar Ahorcado

sin código

Ya diseñaste e implementaste Tic Tac Toe. Ahora toca el segundo juego. Pero esta vez no empezamos de cero: comparamos, reutilizamos y adaptamos.

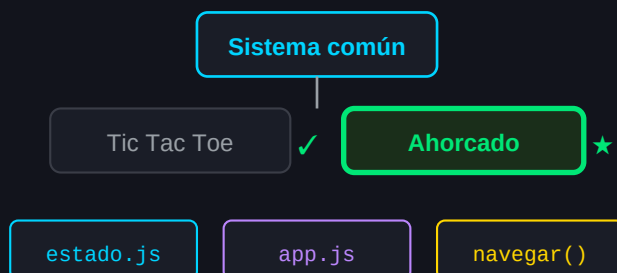
"El segundo módulo demuestra si tu sistema funciona de verdad."

IDEA CENTRAL

Diseñar Ahorcado no es repetir Tic Tac Toe con letras. Es identificar qué cambia, qué se reutiliza y qué decisiones de diseño son diferentes — antes de escribir una sola línea de código.

Dónde estamos en el sistema

APP (sistema)



1 Comparar antes de diseñar

Un buen ingeniero no empieza cada módulo desde cero. Primero mira qué ya tiene y qué puede reutilizar. Después identifica qué es diferente.

Aspecto	Tic Tac Toe	Ahorcado
Jugadores	2 (turnos)	1 (contra el sistema)
Tablero	3x3 fijo	Palabra oculta (variable)
Input	Elegir celda	Elegir letra
Progreso	Visible siempre	Se revela poco a poco
Fin del juego	3 en raya o empate	Palabra o errores max
Errores	Movimiento inválido	Letra incorrecta = daño
Estado clave	tablero[]	palabra, letrasUsadas, errores

Observación clave: el patrón **evento** → **estado** → **UI** no cambia. Lo que cambia es *qué datos* tiene el estado y *qué eventos* dispara el usuario.

2 Qué se reutiliza del sistema

Todo esto ya existe y funciona para Tic Tac Toe. Ahorcado lo usa *sin modificar*:

- **navegar()** — para ir al juego y volver al menú
- **actualizarUI()** — el sistema de renderizado
- **estado.pantallaActual** — la navegación por estado
- **index.html** — se añade un nuevo `<div id="pantalla-juego-ahorcado">`
- **styles.css** — estilos base del sistema

"Si tu sistema está bien diseñado, añadir un segundo juego es extender, no reescribir."

3 Diseñar el estado de Ahorcado

El estado es siempre la primera decisión de diseño. ¿Qué información necesita el sistema para saber cómo está la partida en cualquier momento?

Pseudocódigo — estado del juego

```
// Estado específico de Ahorcado
```

```
estadoAhorcado = {  
  palabraSecreta: "SISTEMA",  
  letrasUsadas: [],  
  erroresMaximos: 6,  
  erroresActuales: 0,  
  letrasCorrectas: [],  
  juegoTerminado: false,  
  haGanado: false  
}
```

Pregunta de diseño: ¿Por qué separamos `letrasCorrectas` de `letrasUsadas`?

Porque cumplen responsabilidades distintas. `letrasUsadas` evita repeticiones. `letrasCorrectas` construye la palabra visible. Mezclarlas sería perder claridad.

4 Reglas en pseudocódigo

Antes de programar, escribimos las reglas como las explicaríamos a otra persona.

Pseudocódigo — lógica de una jugada

CUANDO el jugador elige una letra:

SI la letra ya se usó:

→ no hacer nada (o avisar)

SI la letra está en la palabra:

→ añadir a letrasCorrectas

→ marcar posiciones reveladas

SI la letra NO está en la palabra:

→ sumar 1 a erroresActuales

añadir letra a letrasUsadas

SI erroresActuales >= erroresMaximos:

→ juegoTerminado = true, haGanado = false

SI todas las letras reveladas:

→ juegoTerminado = true, haGanado = true

5 Casos borde

Los casos borde son las situaciones raras o extremas que rompen un juego si no las piensas antes. Un ingeniero los busca *antes* de programar.

¿Qué pasa si la palabra tiene letras repetidas?

Ejemplo: "ARROZ". Si el jugador dice "R", ambas R se revelan. El estado debe reflejar todas las posiciones, no solo la primera.

¿Qué pasa si el jugador introduce algo que no es una letra?

Números, espacios, símbolos. Hay que validar la entrada antes de procesarla.

¿Mayúsculas y minúsculas?

Decisión de diseño: ¿"a" y "A" son lo mismo? Normalmente sí. Convertir todo a mayúsculas antes de comparar.

¿Qué pasa si la palabra tiene espacios o acentos?

Ejemplo: una frase en vez de una palabra. Decisión: por ahora, solo palabras simples sin acentos.

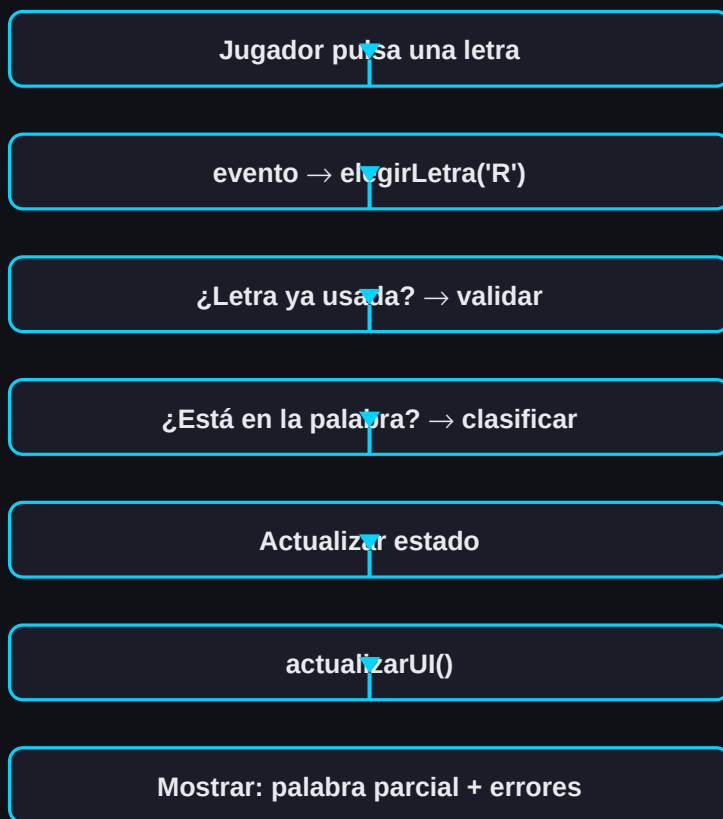
¿Cuántos errores antes de perder?

6 es lo clásico (cabeza, cuerpo, 2 brazos, 2 piernas). Pero es una decisión de diseño — podríamos elegir otro número.

"Los bugs más difíciles nacen de los casos borde que nadie pensó."

6 Flujo del juego: evento → estado → UI

El patrón es el mismo que en Tic Tac Toe. Lo que cambia es el contenido de cada paso.



Diferencia con Tic Tac Toe: en TTT, el evento es "click en celda". En Ahorcado, el evento es "elegir letra". Pero la *estructura* del flujo es idéntica.

7 Diseño de la interfaz

¿Qué necesita ver el jugador en pantalla?

- La palabra parcial: `_ I _ _ E _ A`
- Las letras ya usadas (para no repetir)
- El contador de errores (o el dibujo del ahorcado)
- Los botones de letras (A-Z)
- Un mensaje de victoria o derrota
- Botón de volver al menú

Decisión de diseño: ¿Mostramos un dibujo del ahorcado o solo un número de errores? El dibujo es más visual, pero el número es más simple de implementar. Empieza con el número. El dibujo es una mejora posterior.

8 Qué NO hacer

- No mezclar lógica de Ahorcado con la de Tic Tac Toe en el mismo archivo.
- No poner la palabra secreta visible en el HTML.
- No ignorar las letras repetidas — el jugador las va a pulsar.
- No "hardcodear" una sola palabra — diseña para que sea fácil cambiarla.
- No empezar a programar sin tener el pseudocódigo claro.

9 Uso de IA en esta sesión

Buen prompt:

Prompt que pide criterio, no código

```
"Tengo un juego de Ahorcado. El estado tiene:  
palabraSecreta, letrasUsadas, erroresActuales.  
¿Qué casos borde debería considerar  
antes de programar la lógica?"
```

Mal prompt:

Prompt que delega todo

```
"Hazme un juego de Ahorcado completo  
en JavaScript"
```

La IA puede ayudarte a encontrar casos borde que no habías pensado. Pero el diseño del estado y las reglas son tu responsabilidad.

10 Mini-reto de la sesión

Sin escribir código, entrega:

1. El objeto `estadoAhorcado` con todas sus propiedades justificadas.
2. El pseudocódigo de `elegirLetra(letra)` — la función principal del juego.
3. Al menos 3 casos borde identificados con su solución propuesta.
4. Un boceto (papel o digital) de la pantalla del juego.

**"Si tu diseño funciona en papel,
programarlo es solo traducir."**